



SRE

— FOR —

COMPLETE BEGINNERS



RELIABILITY



MONITORING



AVAILABILITY



AUTOMATION

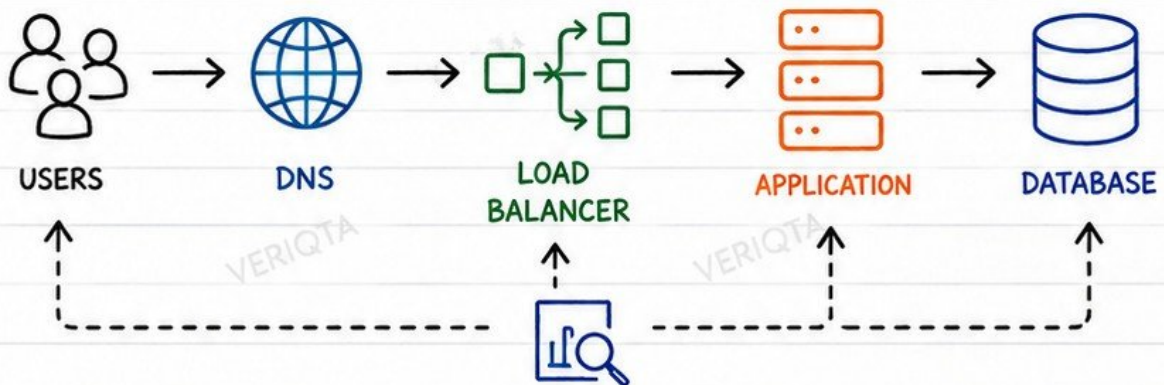


ALERTING



OPERATIONS

YOUR FIRST STEP INTO SITE RELIABILITY ENGINEERING



MONITOR • ALERT • RESPOND • IMPROVE

BUILD RELIABLE
SYSTEMS.
SOLVE REAL
PROBLEMS.
MAKE AN IMPACT.

- ✓ UNDERSTAND RELIABILITY
- ✓ MONITOR LIKE A PRO
- ✓ RESPOND TO INCIDENTS
- ✓ AUTOMATE AND IMPROVE
- ✓ THINK LIKE AN SRE

RELIABILITY
IS NOT A
FEATURE.
IT'S A
PRACTICE.

1. WHAT DOES AN SRE ACTUALLY DO?

SITE RELIABILITY ENGINEERING EXPLAINED SIMPLY

SRE is about making systems reliable, scalable and fast – while automating everything that can be automated. It combines software engineering skills with operations experience to keep services running and users happy.

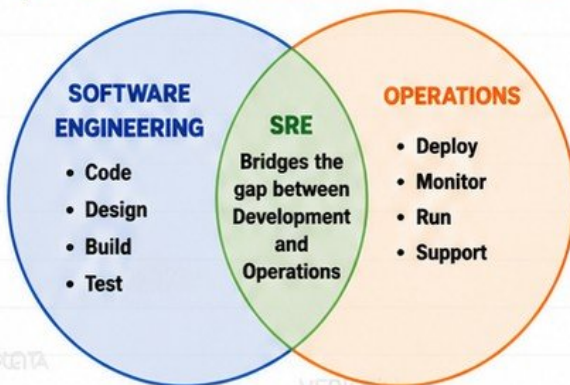
SRE MINDSET

- Reliability is a feature, not an accident
- Measure everything, guess nothing
- Automate to reduce human error
- Learn from failures and improve
- Focus on user experience and system health

SRE vs DEVOPS vs PLATFORM ENGINEERING

ASPECT	SRE	DEVOPS	PLATFORM ENGINEERING
Focus	Reliability of production systems	Faster delivery through collaboration	Building internal developer platforms
Goal	Keep systems reliable, scalable and available	Deliver code to production quickly and safely	Make developer experience better and more productive
Who Uses It	End users	Operations + Development teams	Developers (internal customers)
Key Area	Reliability Engineering	Process + Automation + Culture	Self-service tools, APIs, Developer workflows
Time Spent	Monitoring, incidents, automation, improvements	CI/CD, automation, collaboration	Building platforms, tools, guardrails

SOFTWARE ENGINEERING + OPERATIONS



★ SRE brings engineering discipline to operations problems at scale.

TYPICAL SRE RESPONSIBILITIES

- | | |
|---|--|
|  Monitor systems and services
 Respond to incidents
 Troubleshoot and fix issues
 Improve system reliability |  Automate manual tasks
 Manage alerts and dashboards
 Plan capacity and performance
 Work with developers and other teams |
|---|--|

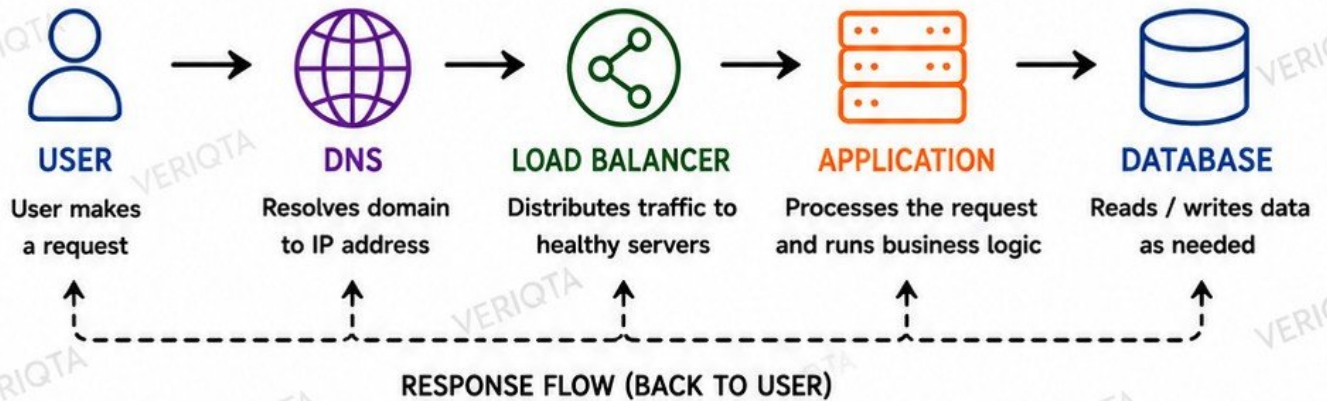
WHAT A JUNIOR SRE MAY ACTUALLY WORK ON

- | | |
|--|---|
| <ul style="list-style-type: none"> ✓ Monitoring dashboards ✓ Alert setup and tuning ✓ Log analysis ✓ Basic troubleshooting | <ul style="list-style-type: none"> ✓ Running operational tasks ✓ Writing scripts/automations ✓ Updating documentation ✓ Supporting on-call rotation |
|--|---|

KEY TAKEAWAY

SREs build reliable systems, operate them, improve them and automate everything possible.

2. HOW A PRODUCTION SYSTEM WORKS



REQUESTS AND RESPONSES

- The user sends a request (e.g., open a website).
- The system processes the request step by step.
- The application may fetch or store data.
- The response travels back the same path to the user.



FRONTEND VS BACKEND

FRONTEND

- What users see
- Runs in the browser
- HTML, CSS, JavaScript
- Communicates with backend via API

BACKEND

- Handles logic and data
- Runs on servers
- Processes requests
- Talks to databases, services, and APIs



SERVERS AND SERVICES

- Servers host applications and services.
- Services perform specific tasks.
- Each service has a responsibility.
- Services communicate over a network.
- More services = more moving parts.



DEPENDENCIES

- Applications depend on other services.
- If a dependency fails, your app can fail.
- Common dependencies: DNS, Load Balancer, Database, APIs, Cache, Message Queue, etc.
- Manage, monitor, and design for failure.



WHERE PRODUCTION FAILURES CAN OCCUR



USER / CLIENT

- Poor network
- Wrong device
- Browser issues



DNS

- DNS down
- Wrong records
- Slow resolution



LOAD BALANCER

- LB down
- Misconfiguration
- Unhealthy targets



APPLICATION

- App errors
- High CPU/RAM
- Code bugs



DATABASE

- DB down
- Slow queries
- Connection limits



EXTERNAL SERVICES

- API failures
- Third-party down
- Timeouts



KEY TAKEAWAYS

- ✓ A production system is a chain of dependent services.
- ✓ A failure in any part can affect the whole system.
- ✓ Monitor every layer.
- ✓ Design for reliability and recovery.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

3. LINUX FUNDAMENTALS FOR SRE



PROCESSES

Programs running on your system.



CPU

The brain. Executes instructions.



MEMORY

Stores data used by running processes.



DISK

Long-term storage for your data.



FILESYSTEMS

Organize and structure data on disk.



SERVICES

Background processes that keep things running.



PERMISSIONS

Control who can read, write, or execute files.



LOGS

Record events useful for troubleshooting.

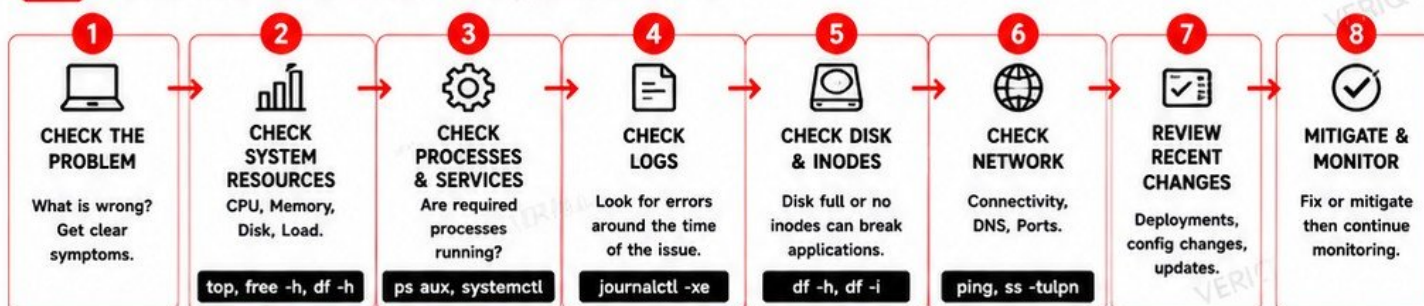


ESSENTIAL LINUX COMMANDS EVERY SRE SHOULD KNOW

COMMAND	WHAT IT DOES	EXAMPLE	SAMPLE OUTPUT (EXAMPLE)
ps aux	Show running processes	ps aux	USER PID %CPU %MEM COMMAND root 1 0.0 0.1 systemd nginx 1234 0.1 1.2 nginx: worker
top	Real-time view of system processes	top	Tasks: 123 total, 1 running %Cpu(s): 2.0 us, 1.0 sy, 0.0 ni, 96.0 id MiB Mem : 3940.0 total, 1200.0 free
free -h	Show memory usage	free -h	total used free shared buff/cache available Mem: 3.8G 1.6G 1.2G 100M 1.0G 2.0G Swap: 2.0G 0B 2.0G
df -h	Show disk space usage	df -h	Filesystem Size Used Avail Use% Mounted on /dev/sda1 50G 20G 28G 42% / /dev/sdb1 100G 10G 90G 10% /data
du -sh *	Show size of files/directories in current path	du -sh *	120M logs/ 2.4G app/ 512K config/
systemctl status nginx	Check status of a service	systemctl status nginx	● nginx.service - A high performance web server Active: active (running) since Wed 10:20 Main PID: 1234 (nginx)
journalctl -xe	View system logs (detailed)	journalctl -xe	May 15 10:20:15 server systemd[1]: Started nginx May 15 10:21:02 server nginx[1234]: error connect() failed



FIRST-RESPONSE LINUX TROUBLESHOOTING



QUICK REFERENCE CHEAT SHEET

- High CPU → `top`
- High Memory → `free -h`
- Disk Almost Full → `df -h`
- Large Directory → `du -sh *`
- Service Not Running → `systemctl status <service>`

- View Logs → `journalctl -xe`
- Follow Logs Live → `journalctl -f`
- Check Listening Ports → `ss -tulpn`
- Check Failed Logins → `journalctl | grep "Failed password"`
- Reboot Required? → `uptime`



KEY TAKEAWAY

Understand the system, use the right command, check the logs, and solve one layer at a time.



SRE MINDSET

Observe → Understand → Hypothesize → Test → Fix → Verify → Learn



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

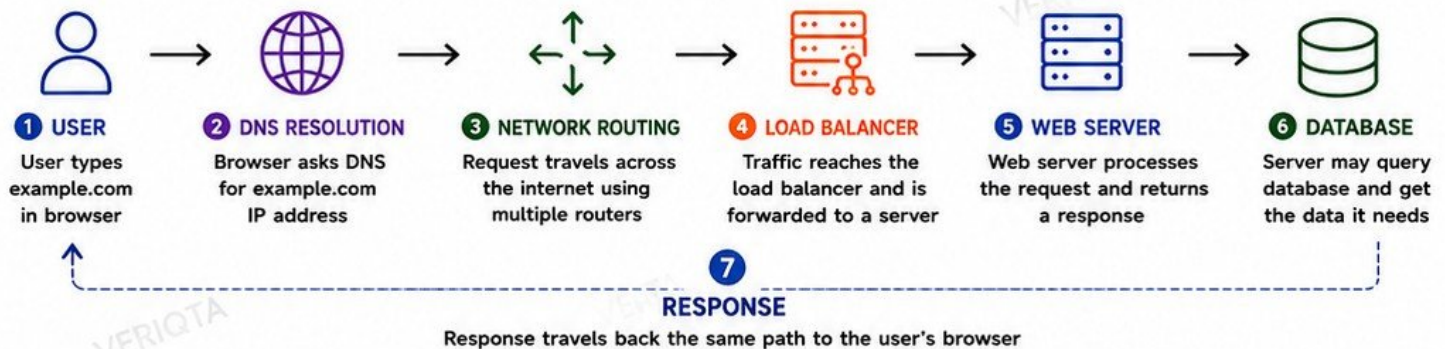
4. NETWORKING EVERY SRE MUST UNDERSTAND

 IP ADDRESSES Unique identifiers for devices on a network. (IPv4 / IPv6) Example: 192.168.1.10	 PORTS Logical endpoints on a server. Services listen on specific ports. Example: 80, 443, 22	 TCP Connection-oriented protocol. Ensures reliable delivery of data. Reliable & Ordered	 DNS Translates domain names (e.g., example.com) to IP addresses. Example: 93.184.216.34	 HTTP / HTTPS HTTP is for web traffic. HTTPS is HTTP secured with TLS. HTTP (80) HTTPS (443)	 TLS Encrypts data in transit. HTTPS uses TLS to keep traffic secure. Secure Encryption	 ROUTING Moves packets from source to destination using best paths. Best Path Selection	 FIREWALLS Allow or block traffic based on rules for security and control. Allow / Block Traffic
---	--	---	---	---	--	--	---

ESSENTIAL NETWORK COMMANDS

COMMAND	WHAT IT DOES	EXAMPLE	SAMPLE OUTPUT (EXAMPLE)
ping <host>	Checks connectivity to a host using ICMP.	ping 8.8.8.8	64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=20.3 ms 64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=20.1 ms
curl <url>	Makes an HTTP/HTTPS request and shows response.	curl -I https://example.com	HTTP/2 200 content-type: text/html content-length: 1256
dig <domain>	Queries DNS and shows detailed DNS records.	dig example.com	;; ANSWER SECTION: example.com. 300 IN A 93.184.216.34
ss -tulinp	Shows listening ports and network connections.	ss -tulinp	Netid State Recv-Q Send-Q Local Address:Port tcp LISTEN 0 128 0.0.0.0:22
traceroute <host>	Shows the path packets take to reach the destination.	traceroute 8.8.8.8	1 192.168.1.1 1.123 ms 2 10.0.0.1 2.456 ms 3 8.8.8.8 20.321 ms

FOLLOWING A REQUEST ACROSS THE NETWORK



 COMMON PORTS 22 SSH (Secure Shell) 80 HTTP 443 HTTPS 53 DNS 3306 MySQL 5432 PostgreSQL	 WHERE NETWORK ISSUES HAPPEN - DNS not resolving - Port blocked by firewall - Service not listening on expected port - Wrong IP or routing - High latency or packet loss - TLS/SSL certificate problems
 KEY TAKEAWAYS - A request goes through many layers before reaching the application. - Network tools help you see what is happening at each step. - Understanding networking helps you find and fix issues faster.	

5. UNDERSTANDING PRODUCTION AVAILABILITY



WHAT "AVAILABLE" ACTUALLY MEANS

A system is **AVAILABLE** when it is working and accessible to users as expected. It's not just "up". It must also be performing its function.



UPTIME VS AVAILABILITY

UPTIME

- Measures how long a system is running.
- Does not include planned downtime.
- Can be misleading.

AVAILABILITY

- Measures how often a system is usable by users.
- Includes both planned and unplanned downtime.



AVAILABILITY PERCENTAGES EXPLAINED

AVAILABILITY	DOWNTIME PER YEAR	DOWNTIME PER MONTH	DOWNTIME PER DAY
99%	3.65 DAYS	7H 18M	14M 24S
99.9%	8H 45M	43M 50S	1M 26S
99.99%	52M 34S	4M 23S	8.6S
99.999%	5M 15S	26S	0.86S

Higher availability = Less downtime = More reliability



WHY 100% AVAILABILITY IS UNREALISTIC

- Hardware fails.
- Software has bugs.
- Networks go down.
- Power outages happen.
- Security updates require restarts.
- Human errors occur.
- External services can fail.

The goal is **NOT 100%**.
The goal is **ACCEPTABLE RISK** with **HIGH RELIABILITY**.



PLANNED VS UNPLANNED DOWNTIME

PLANNED DOWNTIME

- Scheduled maintenance
- Deployments
- System updates
- Announced in advance
- Should be minimized and communicated

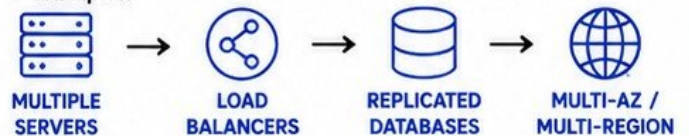
UNPLANNED DOWNTIME

- Unexpected failures
- Crashes
- Outages
- Not announced
- Directly impacts users
- Must be detected and resolved quickly



REDUNDANCY

- Duplicate critical components.
- No single component failure should bring down the system.
- Examples:



SINGLE POINTS OF FAILURE (SPOF)

- A single component whose failure stops the entire system.
- Examples:



SINGLE SERVER



SINGLE DATABASE



SINGLE DNS



SINGLE POWER SOURCE

ELIMINATE SPOFs TO IMPROVE RELIABILITY.



AVAILABILITY CALCULATION EXAMPLE

Formula:

$$\text{AVAILABILITY (\%)} = \frac{\text{TOTAL TIME} - \text{DOWNTIME}}{\text{TOTAL TIME}} \times 100$$

Example:

- Total time in a month = 30 days = 43,200 minutes
- Downtime = 60 minutes

$$\text{Availability} = \frac{43,200 - 60}{43,200} \times 100 = 99.86\%$$



KEY TAKEAWAY

- ✓ Understand availability, not just uptime.
- ✓ Design for failure, not for perfection.

- ✓ Reduce downtime. Respond fast. Learn and improve.
- ✓ Reliable systems build trust and keep users happy.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

veriqta.com

6. SLI, SLO, AND SLA WITHOUT THE CONFUSION



SERVICE LEVEL INDICATOR (SLI)

A MEASURABLE SIGNAL THAT SHOWS HOW YOUR SERVICE IS PERFORMING.

It answers:
"HOW ARE WE DOING?"

Examples:

- Availability
- Latency (Response Time)
- Error Rate (Failures)
- Throughput
- Saturation (CPU, Memory, Disk)



SERVICE LEVEL OBJECTIVE (SLO)

A TARGET FOR YOUR SLI. IT DEFINES THE LEVEL OF RELIABILITY YOU AIM TO ACHIEVE.

It answers:
"WHAT LEVEL OF PERFORMANCE DO WE COMMIT TO?"

Examples:

- Availability SLO: 99.9%
- Latency SLO: 95% of requests < 300ms
- Error Rate SLO: < 0.1% of requests fail



SERVICE LEVEL AGREEMENT (SLA)

A PROMISE YOU MAKE TO YOUR USERS OR CUSTOMERS.

It answers:
"WHAT DO WE PROMISE OUR USERS?"

Examples:

- "Our service will be available 99.9% of the time each month."
- "We will respond to issues within 1 hour."

COMMON SLIS EVERY SRE SHOULD KNOW			
SLI TYPE	WHAT IT MEASURES	EXAMPLE METRIC	EXAMPLE FORMULA
 AVAILABILITY SLI	Whether the service is available and accessible.	successful_requests / total_requests	Availability (%) = $\frac{\text{successful_requests}}{\text{total_requests}} \times 100$
 LATENCY SLI	How fast the service responds.	response_time_seconds (95th percentile)	Latency SLI = 95% of requests completed in < X ms
 ERROR-RATE SLI	How often requests are failing.	failed_requests / total_requests	Error Rate (%) = $\frac{\text{failed_requests}}{\text{total_requests}} \times 100$



CHOOSING MEANINGFUL INDICATORS

- ✓ Focus on what matters to users.
- ✓ Keep it simple and measurable.
- ✓ Must be collected consistently.
- ✓ Should reflect real user experience.
- ✓ Avoid vanity metrics (e.g., page views).
- ✓ Ensure it can drive action when violated.

SLI → SLO → SLA RELATIONSHIP



SLI MEASURE
What is happening?

Raw data and signals from your system.



SLO TARGET
What level of performance do we aim for?

Set internal goals for reliability.



SLA PROMISE
What do we promise users or customers?

Commitments published in contracts or docs.

PRACTICAL EXAMPLES			
SERVICE	SLI (WHAT WE MEASURE)	SLO (OUR TARGET)	SLA (OUR PROMISE)
 E-COMMERCE WEBSITE	Availability (%)	99.9% each month	We guarantee 99.9% availability for our users each month.
 PAYMENT API	Latency (95th percentile)	95% of requests < 300ms	95% of requests will complete within 300ms.
 FILE UPLOAD SERVICE	Error Rate (%)	< 0.1% errors	Less than 0.1% of uploads may fail.



KEY TAKEAWAY

You can't improve what you don't measure (SLI).
You can't manage what you don't target (SLO).
You can't commit what you can't deliver (SLA).

GOOD SLOs ARE:

- ✓ Realistic
- ✓ Measurable
- ✓ Actionable
- ✓ Understandable
- ✓ Aligned with user experience
- ✓ Revisited regularly

7. ERROR BUDGETS



WHAT IS AN ERROR BUDGET?

The error budget is the amount of unreliability you allow while still meeting your SLO.

In simple terms:

It is the "room to fail" without breaking your promise.



WHY SRE TEAMS USE THEM

- Balance reliability and speed.
- Prevent chasing 100% reliability.
- Make reliability a shared responsibility.
- Create data-driven decisions.
- Protect users while allowing innovation.



SLO → ALLOWABLE FAILURE

- The SLO defines the target.
- The error budget is the allowable failure.
- If you stay within the budget, you meet the SLO.
- If you exceed it, you violate the SLO.



RELIABILITY VS RELEASE VELOCITY



Error budgets allow teams to ship fast without sacrificing reliability.
You can go fast... within the budget.



CONSUMING THE ERROR BUDGET

- Failures, slow responses, and errors consume budget.
- The more issues users feel, the faster the budget drains.
- Every incident has a cost against the budget.



WHAT HAPPENS WHEN THE BUDGET IS EXHAUSTED?

- You have violated your SLO.
- Time to focus on reliability, not new features.
- Slower or frozen releases.
- More testing and hardening.
- Fix root causes.
- Rebuild the budget.

NO BUDGET LEFT = NO MORE RISKY CHANGES



SIMPLE ERROR-BUDGET CALCULATION

STEP 1: DEFINE YOUR SLO

Example SLO:
99.9% availability over 30 days
Downtime allowed = 0.1%

STEP 2: CALCULATE ALLOWABLE DOWNTIME

Total time in 30 days = $30 \times 24 \times 60 = 43,200$ minutes
Downtime allowed (0.1%) = $43,200 \times 0.001 = 43.2$ minutes

You have 43.2 minutes of downtime as your error budget.

STEP 3: TRACK CONSUMPTION

If 10 minutes of downtime happens during the 30 days:

Budget used = 10 minutes
Budget remaining = $43.2 - 10 = 33.2$ minutes



EXAMPLE: 99.9% AVAILABILITY SLO

PERIOD	TOTAL TIME	SLO	ALLOWABLE DOWNTIME (ERROR BUDGET)	EXAMPLE DOWNTIME	BUDGET REMAINING	BUDGET USED
7 DAYS	10,080 minutes	99.9%	10.08 minutes	2.50 minutes	7.58 minutes	24.8%
30 DAYS	43,200 minutes	99.9%	43.20 minutes	15.00 minutes	28.20 minutes	34.7%
90 DAYS	129,600 minutes	99.9%	129.60 minutes	40.00 minutes	89.60 minutes	30.9%



KEY TAKEAWAY

Error budgets give teams the freedom to innovate while keeping reliability promises. Use the budget wisely, learn from failures, and protect the user experience.



TIPS TO MANAGE ERROR BUDGETS

- ✓ Monitor SLOs and budget in real time.
- ✓ Review budget in every release decision.
- ✓ Automate alerts when budget is low.
- ✓ Invest in reliability to earn back more budget.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

8. MONITORING FUNDAMENTALS



WHY PRODUCTION SYSTEMS NEED MONITORING

- Detect problems before users are impacted.
- Understand system health and performance.
- Troubleshoot issues faster.
- Ensure availability, performance, and reliability.
- Make data-driven decisions.
- Prove that the system meets its SLOs.

THE 5 PILLARS OF OBSERVABILITY



METRICS

Numeric measurements over time. Show trends and patterns.



LOGS

Discrete records of what happened. Used for investigation.



TRACES

Follow a request as it travels across services. Show the full journey.



EVENTS

Important state changes or occurrences. Useful for alerting.



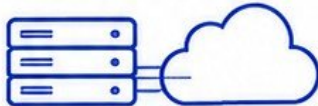
DEPENDENCIES

Your system relies on other services. Monitor them too.



INFRASTRUCTURE MONITORING

- Monitor the underlying resources.
- Examples:
 - CPU, Memory, Disk, Network
 - Servers, Containers, Kubernetes
 - Databases, Load Balancers, DNS
 - Cloud resources and quotas
- Helps you know if the platform can support your application.



APPLICATION MONITORING

- Monitor how the application behaves.
- Examples:
 - Request rate and latency
 - Error rate (4xx, 5xx)
 - Throughput and response time
 - Business metrics (signups, orders)
- Helps you know if your application is healthy from the user's view.



BLACK-BOX VS WHITE-BOX

BLACK-BOX MONITORING (External)

- Monitors from outside your system.
- Simulates a real user.
- Checks availability and responses.
- Example: Ping, HTTP checks, synthetic monitoring.

WHITE-BOX MONITORING (Internal)

- Monitors internal components.
- Uses metrics, logs, traces.
- Gives deep visibility into internals.
- Example: Prometheus, Logs, Traces.

✓ WHAT SHOULD ACTUALLY BE MONITORED?



USER EXPERIENCE

Is the system fast, available, and working for users? (Uptime, Latency, Errors)



SYSTEM HEALTH

Are the components healthy and stable? (CPU, Memory, Disk, Network, Dependencies)



APPLICATION HEALTH

Is the application processing requests correctly? (Error rate, Success rate, Throughput)



BUSINESS HEALTH

Is the business achieving its goals? (Signups, Orders, Payments, Active Users)



SECURITY & SAFETY

Is anything suspicious or risky happening? (Auth failures, Traffic spikes, Config changes)



COST & CAPACITY

Are we using resources efficiently? (Cost, Usage, Capacity trends)



WHAT GOOD MONITORING IS NOT

- Not collecting every metric.
- Not creating alerts for everything.
- Not dashboards with hundreds of charts.
- Not alerts with no clear action.
- Focus on what matters, keep it simple.



MONITORING PRINCIPLES

- Monitor for symptoms, alert on causes.
- Keep it simple and actionable.
- Reduce noise, increase signal.
- Automate where possible.
- Review and improve continuously.



KEY TAKEAWAY

Monitoring is how you understand your system. The right visibility helps you detect issues early, respond fast, and build reliable systems users can trust.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

veriqta.com

9. THE FOUR GOLDEN SIGNALS

THE HEALTH OF YOUR SYSTEM AT A GLANCE

1. LATENCY



HOW LONG REQUESTS TAKE

PRACTICAL EXAMPLE

The time it takes for a user to load a page.

WHAT IT REVEALS

- Slowness in the system
- Poor user experience
- Backend or dependency issues

2. TRAFFIC



HOW MUCH LOAD YOUR SYSTEM HAS

PRACTICAL EXAMPLE

Number of requests per second.

WHAT IT REVEALS

- Usage patterns
- Growth or spikes
- Capacity planning needs

3. ERRORS



THE RATE OF FAILURES

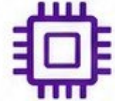
PRACTICAL EXAMPLE

% of requests that fail (4xx, 5xx).

WHAT IT REVEALS

- Broken functionality
- User impact
- Code or dependency problems

4. SATURATION



HOW FULL YOUR SYSTEM RESOURCES ARE

PRACTICAL EXAMPLE

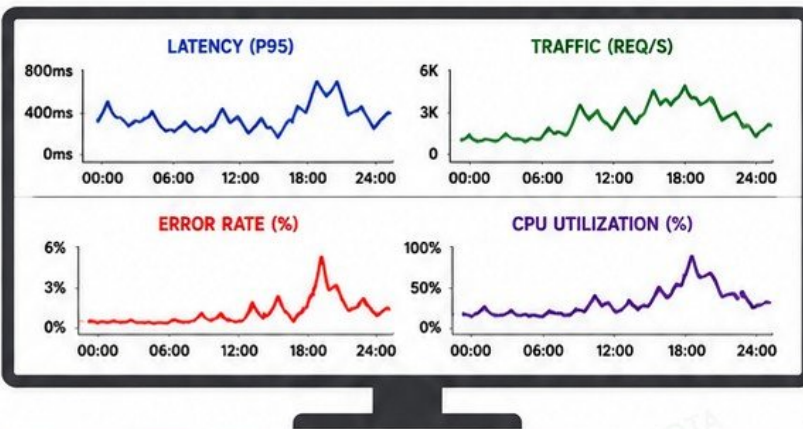
CPU usage, Memory usage, Disk usage.

WHAT IT REVEALS

- Resource exhaustion
- Performance degradation
- Risk of system failure



TURNING SIGNALS INTO USEFUL DASHBOARDS



SIMPLE & CLEAR

One panel per signal. Easy to read.



REAL-TIME VISIBILITY

See trends and detect problems early.



CORRELATE SIGNALS

Use all four together to find root causes.

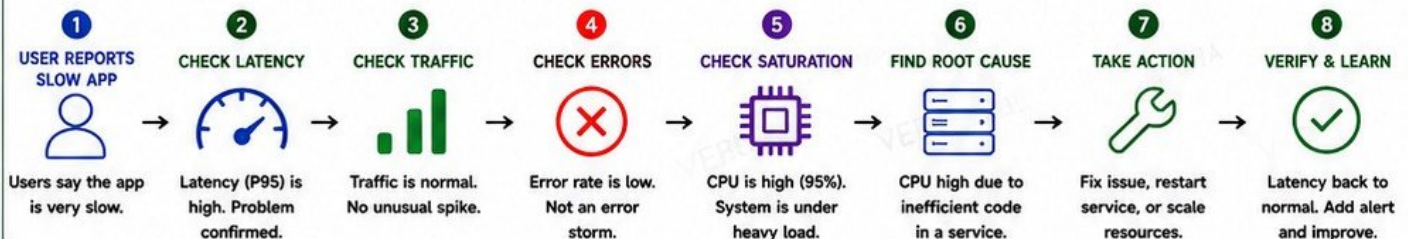


ACTIONABLE ALERTS

Alert only when something needs action.



BEGINNER PRODUCTION INVESTIGATION EXAMPLE



KEY TAKEAWAY

The Four Golden Signals give you a simple but powerful way to understand the health of your system. Master them, and you can solve most production issues.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

≡ veriqta.com ≡

10. PROMETHEUS FOR BEGINNERS

OPEN SOURCE MONITORING & ALERTING SYSTEM



WHAT PROMETHEUS DOES

- Collects metrics from configured targets (services, apps, servers).
- Stores time-series data efficiently.
- Powerful query language (PromQL) to explore metrics.
- Works well with dynamic environments like Kubernetes.
- Integrates with Grafana for visualization and Alertmanager for alerts.



TARGETS

- Anything Prometheus monitor.
- Applications
- Databases
- Servers / Node Exporter
- Kubernetes Pods
- Cloud services
- Custom exporters you build



EXPORTERS

- Expose metrics in a format Prometheus understands.
- Popular exporters:
- Node Exporter
- cAdvisor
- Blackbox Exporter
- MySQL Exporter
- Custom Application Exporter



METRICS

- Numeric measurements about your system.
- Examples:
- `http_requests_total`
- `cpu_usage_seconds_total`
- `memory_usage_bytes`
- `request_duration_seconds`
- Always has a name and optional labels.



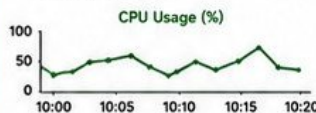
SCRAPING

- Prometheus pulls metrics from targets at a regular interval (scrape interval).
- Default interval: 15s
- Each target exposes a `/metrics` endpoint.
- If a target is down, Prometheus shows it as DOWN.



TIME-SERIES DATA

- Metrics are stored with a timestamp.
- You can see how a metric changes over time.
- Example:
CPU usage at 10:00, 10:05, 10:10...



LABELS

- Key-value pairs that add context to metrics.
- Used to filter, group, and aggregate.
- Example:
`http_requests_total{method="GET", status="200", instance="app-1:8080"}`
- Labels make metrics powerful and flexible.



PROMQL BASICS

- Prometheus Query Language to explore metrics.
- Examples:
- `http_requests_total`
- `rate(http_requests_total[5m])`
- `sum(rate(http_requests_total[5m])) by (method)`
- `http_requests_total{status="500"}`
- Use PromQL in Grafana or Prometheus UI.

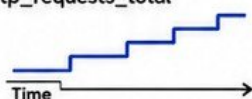
METRIC TYPES



COUNTER

- Only goes up. Resets to 0.
- Used for totals.
- Example:
`http_requests_total`

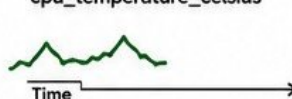
Increasing over time.



GAUGE

- Can go up and down.
- Used for current values.
- Example:
`cpu_temperature_celsius`

Value can fluctuate.



HISTOGRAM

- Tracks distribution of values.
- Used for request durations.
- Example:
`http_request_duration_seconds`

Shows how values are spread.



APPLICATION → METRICS ENDPOINT → PROMETHEUS FLOW



APPLICATION
Exposes `/metrics` endpoint

```
GET /metrics
http_requests_total 1200
cpu_usage_seconds 0.45
memory_usage_bytes 1.2e9
...
```

METRICS ENDPOINT
Plain text format

SCRAPE
(Pull)



PROMETHEUS
Scrapes and stores time-series data

QUERY (PromQL)



GRAFANA / ALERTS
Visualize and create alerts



KEY TAKEAWAY

Prometheus collects metrics from your systems, stores them as time-series data, and helps you understand the health and performance of your infrastructure.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

≡ veriqta.com ≡

11. GRAFANA FOR BEGINNERS



WHAT GRAFANA DOES

Grafana is a visualization and analytics platform. It turns your metrics, logs, and data into beautiful, interactive dashboards so you can understand and monitor your systems in real time.

HOW IT WORKS



1. DATA SOURCES

- Connect Grafana to your data.
- Common sources: Prometheus, Loki, InfluxDB, MySQL, Postgres, CloudWatch.
- Manage credentials securely.
- Test the connection.



2. DASHBOARDS

- A dashboard is a collection of panels.
- Organize by team, service, or environment.
- Share dashboards with your team.
- Use folders and permissions.



3. PANELS

- A panel is a single visualization.
- Many panel types: Graph, Stat, Gauge, Table, Pie chart, Bar chart, Heatmap.
- Each panel shows one metric or value.
- Customize title, unit, color, thresholds.



4. QUERIES

- Queries tell Grafana what data to show.
- Write queries using your data source (e.g., PromQL for Prometheus).
- Build queries visually with the query builder.
- Test queries before adding to panels.



5. VARIABLES

- Variables make dashboards dynamic.
- Common types: Query, Custom, Data source, Interval.
- Examples: \$env, \$region, \$service
- Let users filter dashboards easily.



6. VISUALIZING PROMETHEUS METRICS

- Add Prometheus as a data source.
- Use PromQL to query metrics.
- Example: `rate(http_requests_total[5m])`
- Show trends, rates, errors, latency, traffic.
- Use meaningful labels and group by.



7. BUILDING A USEFUL SRE DASHBOARD

REQUEST RATE

1.25k req/s



ERROR RATE

2.35%



P95 LATENCY

320 ms



CPU USAGE

68%



MEMORY USAGE

72%



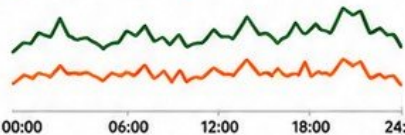
ACTIVE USERS

842



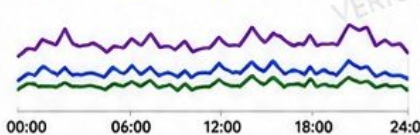
REQUESTS OVER TIME

• Total Requests • Errors



LATENCY OVER TIME (P50, P95, P99)

• P50 • P95 • P99



ERRORS BY SERVICE



Focus on the Four Golden Signals
(Latency, Traffic, Errors, Saturation)



Show RED (problems) quickly



Use thresholds and alerts



Keep it simple and actionable



Include key business metrics if relevant



8. AVOIDING DASHBOARDS FULL OF MEANINGLESS GRAPHS



Don't add metrics just because you can



Avoid too many panels on one screen



Don't duplicate the same metric in different ways



Remove metrics no one uses



Focus on questions the dashboard should answer



Good dashboards help you act, not just look



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

12. ALERTING WITHOUT ALERT FATIGUE



1. WHAT DESERVES AN ALERT

- User impact or risk of impact
- Breach of SLO or error budget
- System is degraded or down
- Security or compliance issue
- Capacity or resource at risk
- Business critical dependency failure



2. WARNING VS CRITICAL

WARNING (Heads Up)

- Early sign of a problem
- Not impacting users yet
- Gives time to prepare

CRITICAL (Immediate Action)

- User impact is happening
- SLO at risk or breached
- Needs urgent response



3. SYMPTOMS VS CAUSES

SYMPTOM (Bad)

- CPU is high
- Disk is full
- Service is down

CAUSE (Good)

- Specific process leaking memory
- Full disk on /var/log
- Database connection pool exhausted



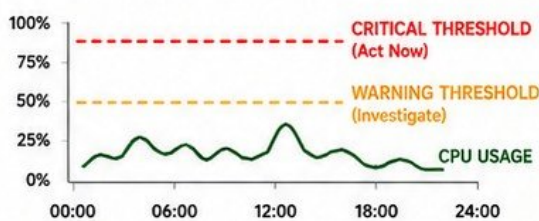
4. ACTIONABLE ALERTS

- Clear and specific
- Include who, what, where
- Tell you what to do next
- Point to dashboards/runbooks
- Avoid generic or noisy alerts
- If you don't act, don't alert



5. THRESHOLD-BASED ALERTS

Alert when a metric crosses a defined threshold.



EXAMPLES

- CPU usage > 85% for 5m
- Error rate > 3% for 2m
- Response time > 1s (p95)
- Disk usage > 90%



6. ALERTMANAGER BASICS

- Receives alerts from Alerting tools (Prometheus, etc.)
- Groups similar alerts
- Deduplicates to reduce noise
- Routes alerts to the right people
- Supports silences and inhibitions
- Ensures alerts are delivered reliably



7. ROUTING AND ESCALATION



BEST PRACTICES

- ✓ Route by severity, service, team, and time.
- ✓ Use quiet hours and maintenance windows.
- ✓ Escalate when not acknowledged.
- ✓ Always include runbook/dashboard links.



8. WHY TOO MANY ALERTS BECOME DANGEROUS

- ✗ Engineers start ignoring alerts.
- ✗ Real problems get missed.
- ✗ On-call becomes stressful and inefficient.
- ✗ Wastes time and reduces trust in monitoring.
- ✗ Leads to slower response and bigger outages.

KEEP ALERTS

- ✓ Few but important
- ✓ Actionable
- ✓ Clear
- ✓ Reliable



9. EXAMPLE OF A USEFUL PRODUCTION ALERT

ALERT NAME:
High Error Rate on Checkout Service

SEVERITY: CRITICAL

DESCRIPTION:
Error rate is 5.2% for 5 minutes

SERVICE: checkout-api

ENVIRONMENT: production

DASHBOARD:
<https://grafana.com/d/checkout-service>

RUNBOOK:
<https://runbooks.com/checkout-errors>

WHY THIS ALERT IS GOOD

- ✓ Specific and clear
- ✓ Includes severity
- ✓ Includes service and environment
- ✓ Includes value and duration
- ✓ Links to dashboard and runbook
- ✓ Actionable and easy to understand



KEY TAKEAWAY

Good alerts tell you what matters, when it matters, and what to do.
The goal is not more alerts, the goal is the **right** alerts.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

≡ veriqta.com ≡

13. LOGS AND PRODUCTION TROUBLESHOOTING

1. WHAT APPLICATION LOGS REVEAL

- What happened in your application
- User actions and requests
- Business events
- Errors and exceptions
- Performance issues
- External API calls
- State changes
- Why something failed

2. SYSTEM LOGS

- Logs from the operating system
- Kernel messages
- Service start/stop
- Authentication events
- Disk / memory issues
- Network problems
- Security events
- Useful locations:

```
/var/log/syslog (or /var/log/messages)
/var/log/auth.log
/var/log/kern.log
journalctl -u <service>
```

3. STRUCTURED LOGS

- Logs in a consistent, machine-readable format
- Common formats: JSON, Logfmt
- Easier to search, filter, and analyze
- Include key fields (timestamp, level, correlation_id, user_id, etc.)
- Example (JSON):

```
{
  "timestamp": "2025-05-21T10:15:30Z",
  "level": "ERROR",
  "service": "checkout",
  "message": "Payment failed",
  "correlation_id": "a1b2c3d4",
  "user_id": "u123",
  "order_id": "o987",
  "status_code": 502
}
```

4. LOG LEVELS

LEVEL	WHEN TO USE
TRACE	Very detailed debugging info
DEBUG	Detailed information for developers
INFO	General operational information
WARN	Something unexpected happened
ERROR	A request or operation failed
FATAL	The system cannot continue

5. TIMESTAMPS

- Every log must have a timestamp
- Use UTC time (not local time)
- ISO 8601 format is best:

```
2025-05-21T10:15:30.123Z
```

- Helps you understand the exact sequence of events

6. REQUEST AND CORRELATION IDS

- Every request should have a unique ID
- Propagate this ID across all services
- Helps you trace one request end-to-end
- Common field names:

```
correlation_id
request_id
trace_id
x-request-id
```

- Example: a1b2c3d4-5678-90ab-cdef-1234567890ab

7. SEARCHING LOGS

COMMON SEARCH METHODS

- Keyword search (error, failed, timeout)
- Filter by log level (level=ERROR)
- Filter by service (service=checkout)
- Filter by time range
- Filter by correlation_id / request_id
- Combine filters

Example (in LogQL / Loki):

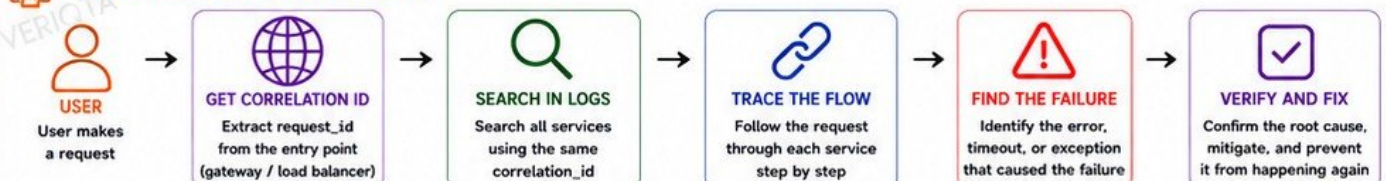
```
{service="checkout", level="ERROR"} |= "payment"
```

USEFUL LOG COMMANDS (LINUX)

- View logs in real time:
- Search error messages:
- Filter by time:
- Filter by service:
- Count errors:

```
tail -f /var/log/syslog
grep -i error /var/log/syslog
journalctl --since "2025-05-21 10:00"
journalctl -u nginx
grep -i error /var/log/app.log | wc -l
```

8. FOLLOWING ONE FAILED REQUEST



9. AVOIDING SECRETS AND SENSITIVE DATA IN LOGS

- Never log passwords, tokens, API keys, or secrets
- Never log full credit card numbers or personal data
- Mask or redact sensitive values
- Follow data privacy and compliance rules (GDPR, PCI-DSS, etc.)
- Example (Bad → Good):

```
Bad: password=MyP@ssw0rd! token=eyJhBgciOiJlUzI1NiIs...
Good: password=**** token=****
```

10. BEST PRACTICES

- ✓ Log what is useful, not everything.
- ✓ Use structured logs.
- ✓ Include timestamp and correlation_id in every log.
- ✓ Use appropriate log levels.
- ✓ Centralize your logs.
- ✓ Keep logs secure and compliant.
- ✓ Delete or archive old logs based on retention policy.

KEY TAKEAWAY

Good logs make problems easy to find and fix.
Clear, structured, and safe logs are your best troubleshooting tool.

14. INCIDENT RESPONSE FOR BEGINNERS



1. WHAT COUNTS AS AN INCIDENT

- Unplanned interruption or degradation of a service
- Impacts users or the business
- Breaches SLOs or error budgets
- Security events
- Anything that needs immediate attention

EXAMPLES

- Website down
- High error rate
- Slow performance
- Data loss or corruption
- Security breach

2. INCIDENT RESPONSE LIFECYCLE



3. SEVERITY LEVELS

SEVERITY	IMPACT	DESCRIPTION	RESPONSE TARGET
SEV 1 - CRITICAL	Major outage, many users or core service down	Full or near-total service unavailable	Respond immediately (0-15 minutes)
SEV 2 - HIGH	Partial outage, significant impact	Important features affected, workarounds limited	Acknowledge within 15-30 minutes
SEV 3 - MEDIUM	Minor impact, workaround available	Some users affected, low business impact	Respond within 1-4 hours
SEV 4 - LOW	Minimal or no user impact	Cosmetic issues, questions, or minor bugs	Respond in normal working time



4. INCIDENT ROLES

- **INCIDENT COMMANDER**
Owns the incident and drives response
- **SRE / ENGINEERS**
Investigate and fix the issue
- **COMMUNICATOR**
Shares updates, reduces noise
- **NOTE TAKER**
Records timeline and decisions
- **STAKEHOLDERS**
Business, support, customers



5. BUILDING AN INCIDENT TIMELINE

10:02	Alert received: High error rate on Checkout Service
10:03	Incident created, SEV 2 declared
10:05	Initial investigation: Errors from Payment API
10:08	Mitigation: Traffic routed to secondary provider
10:12	Error rate dropping, service stabilizing
10:18	Root cause identified: Provider regional outage
10:25	Service recovered, monitoring closely
10:35	Incident resolved, postmortem scheduled



6. FIRST 15 MINUTES OF AN OUTAGE

0-1 MIN	Alert fires → Acknowledge the alert
1-3 MIN	Confirm the issue (not a false alarm)
3-5 MIN	Check dashboards & recent changes
5-8 MIN	Identify blast radius and impact
8-10 MIN	Declare incident and set severity
10-12 MIN	Start mitigation to reduce impact
12-15 MIN	Communicate status and next update

GOALS IN THE FIRST 15 MINUTES

- ✓ Stop the bleeding
- ✓ Understand the impact
- ✓ Keep stakeholders informed



7. DETECTION

- Alerts from monitoring systems
- User reports
- Synthetic tests
- Logs, metrics, and traces
- External monitoring or uptime checks

BEST PRACTICE

Detect fast, alert smart, and avoid noise.



8. TRIAGE

- What is broken?
- What is the impact?
- Who is affected?
- When did it start?
- Any recent changes?
- What looks normal?

BEST PRACTICE

Collect facts first, then act.



9. MITIGATION

- Use runbooks and playbooks
- Rollback recent changes
- Failover to healthy systems
- Scale resources if needed
- Disable failing components
- Communicate what you're doing

BEST PRACTICE

Reduce impact fast, not perfect.



10. RESOLUTION

- Identify root cause
- Fix the problem
- Verify the service is healthy
- Monitor closely after fix
- Restore normal operations

BEST PRACTICE

Fix the cause, not just the symptom.



11. COMMUNICATION

- Be clear and honest
- Share impact and actions
- Give regular updates
- Use the right channels
- Close the loop

BEST PRACTICE

Silence creates fear. Communication builds trust.



12. COMMUNICATION TEMPLATE

Incident: Checkout Service
Severity: SEV 2 - HIGH
Time: 10:05 UTC

Impact: Some users unable to complete checkout.
What we know: Errors from payment provider.
What we are doing: Routing traffic to backup provider.
Next update: In 15 minutes.

Where to follow: #incident-updates
How to reach us: @oncall-team



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

15. ROOT CAUSE ANALYSIS AND POSTMORTEMS



1. SYMPTOM VS ROOT CAUSE

SYMPTOM (What we see)

- The visible problem
- The effect, not the reason
- Example: Users see 502 errors

ROOT CAUSE (Why it happened)

- The underlying reason
- Fixing this prevents recurrence
- Example: Database connection pool exhausted due to a leak



2. ASKING "WHY" SYSTEMATICALLY

Use the 5 Whys Technique



ROOT CAUSE:
Code bug caused connection leak



3. EVIDENCE BEFORE ASSUMPTIONS

- Don't guess, investigate
- Collect logs, metrics, traces
- Check timelines and events
- Validate with data
- Avoid bias and blame
- Facts first, opinions last

GOOD EVIDENCE EXAMPLES

- ✓ Log entries with timestamps
- ✓ Metrics showing resource usage
- ✓ Traces showing request flow
- ✓ Configurations and changes
- ✓ Screenshots and command output

BAD EVIDENCE EXAMPLES

- ✗ "I think..."
- ✗ "Maybe it was..."
- ✗ "It usually happens because..."



4. TIMELINE RECONSTRUCTION

- Rebuild the sequence of events
- Use timestamps from all sources
- Include deployments and changes
- Mark detection and mitigation
- Helps find what happened and when

```

10:02 Deployment started
10:05 First error observed
10:07 Alerts fired
10:10 Incident acknowledged
10:12 Mitigation started
10:25 Service recovered
10:35 Incident resolved
  
```



5. CONTRIBUTING FACTORS

- Not only the root cause, but everything that made it possible
- Technical, process, and human factors
- Example:

Code bug No connection leak tests Missing alerts on pool usage No runbook for this issue High traffic increased load



6. CORRECTIVE ACTIONS

- Fix the root cause
- Improve systems, processes, and tooling
- Actions should be specific and measurable

EXAMPLE ACTIONS

- ✓ Fix connection leak in code
- ✓ Add unit test for connection handling
- ✓ Add alert for high pool usage
- ✓ Update runbook for DB failures
- ✓ Review deployment checklist



7. BLAMELESS POSTMORTEMS

- Focus on systems, not people
- Everyone makes mistakes
- The goal is learning, not blaming
- Create a safe environment
- Encourage honest communication

WE SAY

- ✓ "What happened?"
- ✓ "What can we learn?"
- ✓ "How do we improve?"

WE DON'T SAY

- ✗ "Who is responsible?"
- ✗ "Who made the mistake?"
- ✗ "Why didn't you...?"



8. PREVENTING RECURRENCE

- Learn from every incident
- Improve continuously
- Track action items
- Verify actions are effective
- Share learnings across teams

HOW WE PREVENT RECURRENCE

- ✓ Fix root causes, not just symptoms
- ✓ Improve monitoring and alerts
- ✓ Automate and simplify
- ✓ Update documentation and runbooks
- ✓ Test failure scenarios regularly



9. POSTMORTEM TEMPLATE



1. SUMMARY

- What happened
- When it happened
- Impact



2. TIMELINE

- Key events
- Timestamps
- Detection to resolution



3. IMPACT

- Who was affected
- What was the impact
- Duration



4. ROOT CAUSE

- What caused it
- Why it happened
- Evidence



5. CONTRIBUTING FACTORS

- What made it worse
- System, process, people factors



6. ACTIONS

- What we will do
- Owners
- Due dates



7. WHAT WE LEARNED

- Key takeaways
- How we will avoid this next time



KEY TAKEAWAY

Every incident is an opportunity to build a more reliable system.
Fix the cause. Improve the system. Protect the users.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

16. ON-CALL ENGINEERING

WHAT BEING ON-CALL MEANS

- You are available to respond to incidents outside normal work hours.
- You are the first responder when something breaks.
- Your goal: restore service, reduce impact, and learn from incidents.



PAGER AND ALERT FLOW



Alerts should be actionable, meaningful, and routed to the right person.



ACKNOWLEDGING AN INCIDENT

- Acknowledge quickly.
- Let the team know you are on it.
- Add a short note about what you are seeing.
- Start a timeline document.



CHECKING DASHBOARDS AND LOGS

- Check the relevant dashboards (first).
- Review the Four Golden Signals: Latency, Traffic, Errors, Saturation.
- Look at recent changes (deployments, configs).
- Search logs for errors and patterns.
- Identify the likely failure domain before acting.



RUNBOOKS

- Runbooks are step-by-step guides for common incidents.
- Follow the runbook every time.
- Update runbooks after incidents.
- Runbooks reduce MTTR and prevent mistakes.
- If no runbook exists, document what you did.



ESCALATION

- Escalate when:
 - ✓ You don't know what's happening.
 - ✓ Impact is increasing.
 - ✓ You need access or help.
 - ✓ It's outside your area of responsibility.
- Escalate early, not late.



HANDOVER

- Share a clear summary.
- What's happening?
- What have you done?
- What's the next step?
- Share links: dashboards, logs, runbooks, timeline.
- Confirm the next engineer before signing off.



WHEN NOT TO MAKE RISKY PRODUCTION CHANGES



When you don't fully understand the problem.



When the impact is unclear or still unknown.



When there is a safer way to mitigate first.



During high traffic or critical business hours.



When you don't have a rollback plan.



When you're tired, stressed, or unsure.



Never guess. Always verify first.



BASIC ON-CALL WORKFLOW



ALERT RECEIVED

- Receive alert via pager/app.
- Read the alert carefully.
- Acknowledge immediately.



UNDERSTAND

- Check dashboards.
- Check recent changes.
- Review logs and errors.
- Identify the problem.



MITIGATE

- Apply the safest possible fix.
- Use runbooks if available.
- Reduce impact first.



RESOLVE

- Fix the root cause or apply permanent solution.
- Verify the service is healthy.
- Confirm with team or users if needed.



DOCUMENT

- Update the timeline of actions.
- Note what worked and what didn't.
- Update runbook if needed.



HANDOVER / CLOSE

- Handover if your shift ends.
- Confirm stability.
- Close the incident in the system.



ON-CALL SUCCESS TIPS



Communicate clearly and frequently.



Stay calm and follow the process.



Use data, not assumptions.



Learn from every incident.



Take care of your health and rest.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

17. RELIABILITY THROUGH AUTOMATION



WHY REPETITIVE OPERATIONS SHOULD BE AUTOMATED

- Saves time and reduces manual errors.
- Improves consistency and reliability.
- Allows engineers to focus on meaningful work.
- Enables fast response and recovery.
- Automation is the foundation of scalable operations.



MANUAL TOIL

- Repetitive, manual, time-consuming tasks.
- Usually low value and high effort.
- Prone to mistakes and fatigue.
- Slows down incident response.
- Examples: Restarting services manually, checking servers one by one, copying logs, creating users, clearing cache, etc.



WHAT SRE MEANS BY TOIL

- Work that could be automated.
- Adds no lasting value to the product.
- Necessary today, but should not require human effort every time.
- SRE goal: eliminate, reduce, or automate toil.



BASH AND PYTHON EXAMPLES

BASH: CHECK SERVICE STATUS

```
#!/bin/bash
SERVICE=nginx
if systemctl is-active --quiet $SERVICE; then
  echo "$SERVICE is running"
else
  echo "$SERVICE is not running. Restarting..."
  systemctl restart $SERVICE
fi
```

PYTHON: CHECK HTTP ENDPOINT

```
import requests, sys
url = "https://example.com/health"
try:
  r = requests.get(url, timeout=5)
  if r.status_code == 200:
    print("OK")
    sys.exit(0)
  else:
    print(f"FAIL {r.status_code}")
    sys.exit(1)
except Exception as e:
  print(f"ERROR {e}")
  sys.exit(1)
```



AUTOMATED HEALTH CHECKS

- Continuously verify system health.
- Examples:
 - HTTP endpoint checks
 - Service status checks
 - Disk, CPU, memory checks
 - Dependency checks
- Detect issues early.
- Feed alerts and dashboards.



AUTOMATED REMEDIATION

- Automatically fix issues when safe.
- Examples:
 - Restart a failed service
 - Scale up/down resources
 - Clear stuck jobs
 - Replace unhealthy instances
- Reduces downtime.
- Improves MTTR.



SCHEDULED OPERATIONS

- Run tasks at defined intervals.
- Examples:
 - Log rotation
 - Backups
 - Report generation
 - Certificate renewal
 - Cache cleanup
- Use cron (Linux) or schedulers.
- Keep systems clean and efficient.



TOOLS YOU CAN USE

- Cron / systemd timers
- Ansible / SaltStack
- Bash / Python scripts
- AWS Systems Manager
- Kubernetes CronJobs
- Prometheus + Alertmanager
- ChatOps (Slack/Teams bots)
- Terraform for infrastructure



WHEN AUTOMATION CAN MAKE INCIDENTS WORSE



Bad script can break more things automatically.



Wrong logic can cause restart loops or data loss.



Automated actions may hide the real root cause.



Aggressive automation can overload systems.



No limits or checks = uncontrolled actions.



Automation without logging makes debugging hard.



HUMAN APPROVAL FOR RISKY ACTIONS

- Require approval for high-impact actions.
- Examples: Deleting resources, scaling to zero, database changes, deployments to production.
- Use approval workflows (ChatOps, ITSM tools, or manual gates).



AUTOMATION
DETECTS ISSUE



HUMAN
APPROVAL



ACTION
EXECUTED



LOGGED &
NOTIFIED

- Keep humans in the loop for safety.
- Balance speed with reliability.



KEY TAKEAWAYS



Automate repetitive tasks.



Automate checks and safe fixes.



Schedule routine operations.



Test automation before using it.



Use approvals for risky actions.



Good automation = faster, safer, reliable.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

≡ veriqta.com ≡

18. CONTAINERS AND KUBERNETES FOR SRE

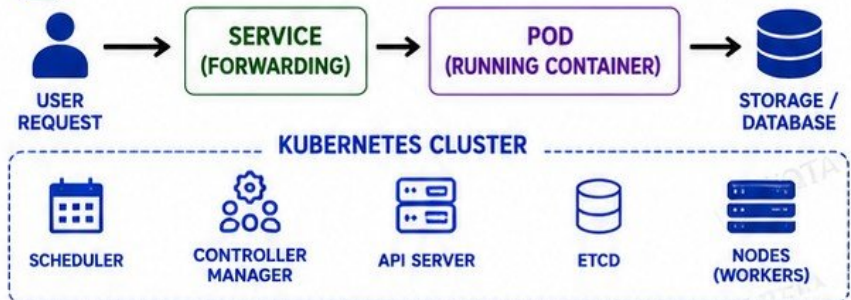


CONTAINERS EXPLAINED SIMPLY

- A container packages your application and everything it needs to run.
- Runs the same way everywhere.
- Lightweight and fast to start.
- Isolated but shares the host OS kernel.
- Docker builds containers.
- Kubernetes runs and manages containers at scale.



HOW KUBERNETES WORKS (HIGH LEVEL)



PODS

- Smallest deployable unit.
- Wraps one or more containers that share the same IP, network, and storage.
- Pods are ephemeral. They can be created, destroyed, and recreated.



DEPLOYMENTS

- Manages ReplicaSets and Pods.
- Declarative way to update applications.
- Supports rolling updates and rollbacks.
- Ensures the desired number of pod replicas are running.



SERVICES

- Stable way to access Pods.
- Provides a constant IP and DNS name.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- Decouples traffic from changing pods.



REPLICAS

- Run multiple copies of a pod for availability.
- Kubernetes replaces failed pods automatically.
- Improves reliability and handles traffic spikes.



READINESS PROBES

- Checks if a pod is ready to receive traffic.
- If not ready, the pod is removed from service endpoints.
- Types: HTTP, TCP, Exec.
- Prevents sending traffic to unready pods.



LIVENESS PROBES

- Checks if a pod is alive.
- If it fails, Kubernetes restarts the container.
- Types: HTTP, TCP, Exec.
- Helps recover from application deadlocks or hangs.



RESOURCE REQUESTS AND LIMITS

- Requests = minimum resources guaranteed.
- Limits = maximum resources a container can use.
- Prevents noisy neighbor issues.
- CPU (cores) and Memory (Mi/Gi).

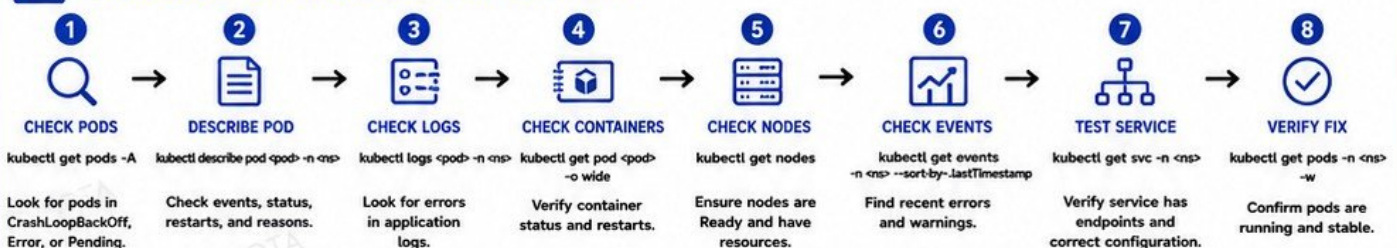


WHY PODS RESTART

- Liveness probe failed.
- Out of memory (OOMKilled).
- Application crash or error.
- Node failure or drain.
- Manual restart / update.
- Configuration or dependency issues.



BASIC KUBECTL TROUBLESHOOTING WORKFLOW



KEY TAKEAWAYS

- ✓ Containers package applications consistently.
- ✓ Kubernetes keeps your applications available.
- ✓ Probes, resources, and replicas improve reliability.
- ✓ Restarts are normal — understand the reason.
- ✓ Use kubectl workflow to diagnose quickly.



COMMON KUBECTL COMMANDS

<code>kubectl get pods -A</code>	# List all pods
<code>kubectl describe pod <pod> -n <ns></code>	# Describe a pod
<code>kubectl logs <pod> -n <ns> -f</code>	# Stream pod logs
<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>	# Show recent events
<code>kubectl top pods -n <ns></code>	# Show resource usage
<code>kubectl rollout status deployment/<name> -n <ns></code>	# Check deployment status



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

18. CONTAINERS AND KUBERNETES FOR SRE



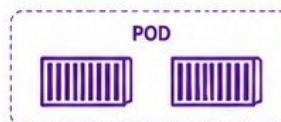
1. CONTAINERS EXPLAINED SIMPLY

- A container packages an application and its dependencies.
- Runs the same everywhere.
- Lightweight and fast to start.
- Many containers run on the same server using less resources.
- Docker builds images, Kubernetes runs them at scale.



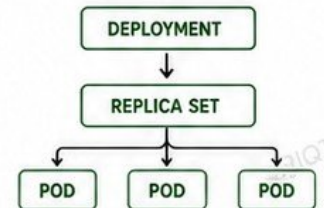
2. PODS

- The smallest deployable unit in Kubernetes.
- One or more containers that share the same:
 - Network (IP address)
 - Storage (volumes)
 - Lifecycle
- Containers in a pod are deployed together.



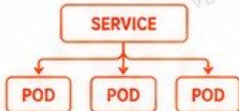
3. DEPLOYMENTS

- Manages replica sets and pods.
- Declarative updates for your app.
- Supports rolling updates and rollbacks.
- Ensures the desired number of pod replicas is running.



4. SERVICES

- Stable IP and DNS for pods.
- Load balances traffic across pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- Decouples clients from pod IPs.



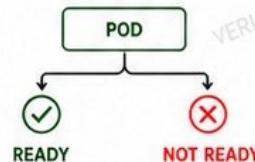
5. REPLICAS

- Number of identical pod copies running.
- More replicas = higher availability.
- Handled automatically by Deployments.
- If a pod fails, a new one is created.



6. READINESS PROBES

- Checks if a pod is ready to receive traffic.
- Example: /ready endpoint.
- If not ready, traffic is stopped from going to the pod.
- Prevents sending traffic to pods still starting.



7. LIVENESS PROBES

- Checks if the container is alive.
- Example: /health endpoint.
- If it fails, Kubernetes restarts the container.
- Helps recover from hangs or deadlocks.



8. RESOURCE REQUESTS AND LIMITS

- Requests: minimum resources needed.
- Limits: maximum resources allowed.
- Protects the cluster from noisy neighbors.
- Helps scheduler place pods correctly.

RESOURCE	REQUEST	LIMIT
CPU	250m	1
MEMORY	256Mi	512Mi

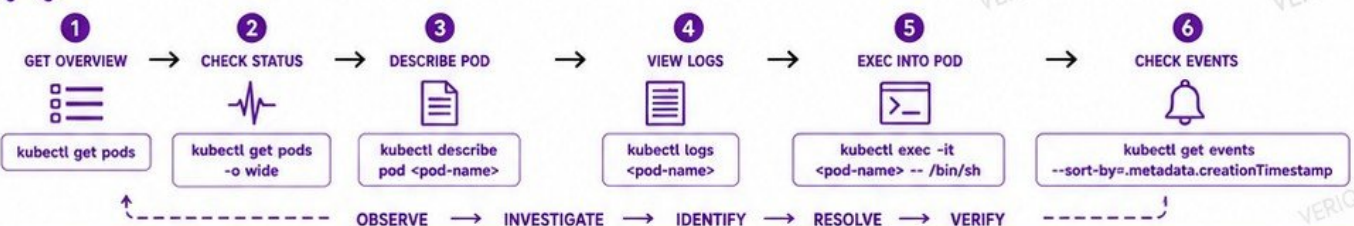


9. WHY PODS RESTART

- Liveness probe failed.
- Application crashed.
- Out of memory (OOMKilled).
- Node failure.
- Manual restart / rolling update.
- Configuration or dependency issues.



10. BASIC kubectl TROUBLESHOOTING WORKFLOW



KEY TAKEAWAYS

- Containers make apps portable.
- Kubernetes runs containers at scale.
- Health probes keep traffic safe.
- Resources protect stability.
- Observe, troubleshoot, and learn.



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

veriqta.com

19. YOUR FIRST PRODUCTION TROUBLESHOOTING SCENARIO



SCENARIO:

"Users report that the application is extremely slow."



GOAL

Find the cause quickly, mitigate the impact, restore normal service, then perform deeper RCA.



PRINCIPLES

- Stay calm and follow the process.
- Verify before you assume.
- Check the system, not just the app.
- Mitigate first, then investigate deeper.



INVESTIGATE - STEP BY STEP

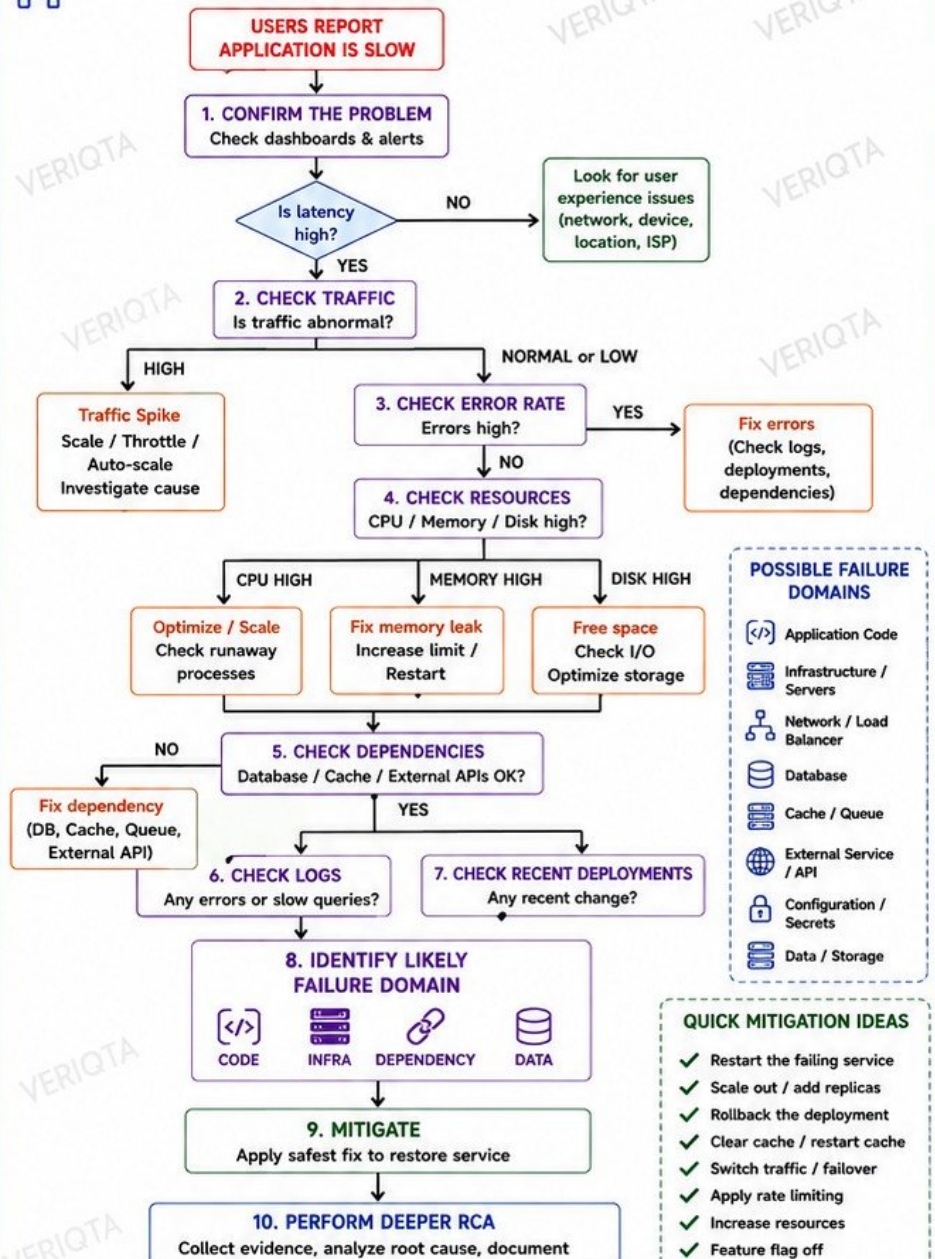
- 1. CONFIRM THE PROBLEM**
Check dashboards, alerts, and user reports.
- 2. CHECK LATENCY**
Is response time high? For which endpoint?
- 3. CHECK TRAFFIC**
Is traffic normal, low, or abnormally high?
- 4. CHECK ERROR RATE**
Are errors increasing? 4xx or 5xx?
- 5. CHECK CPU**
Is CPU usage high on app servers?
- 6. CHECK MEMORY**
Is memory usage high or OOM killing?
- 7. CHECK DISK**
Is disk full or I/O wait high?
- 8. CHECK APPLICATION LOGS**
Look for errors, slow queries, timeouts.
- 9. CHECK DEPENDENCIES**
Database, cache, queues, external APIs.
- 10. CHECK RECENT DEPLOYMENTS**
Any new release or config change?
- 11. IDENTIFY LIKELY FAILURE DOMAIN**
Code, infrastructure, dependency, or data?
- 12. MITIGATE BEFORE DEEPER RCA**
Apply the safest fix to restore service.

TROUBLESHOOTING CHECKLIST

- ✓ Do not guess. Verify with data.
- ✓ Check dashboards before logs.
- ✓ Use time range around the issue.
- ✓ Compare with normal behavior.
- ✓ Look at the full request path.
- ✓ Communicate updates clearly.
- ✓ Mitigate first, perfect later.
- ✓ Document everything.



TROUBLESHOOTING DECISION TREE



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram

20. JUNIOR SRE PRODUCTION PROJECT



YOUR MISSION

Build a small, observable service and operate it like a real production system.

- 1 Deploy a simple application 
- 2 Run it on Linux or containers 
- 3 Expose a health endpoint 
- 4 Collect Prometheus metrics 
- 5 Build a Grafana dashboard 
- 6 Monitor the Four Golden Signals 
- 7 Configure one meaningful alert 
- 8 Simulate an application failure 
- 9 Detect the incident 
- 10 Troubleshoot using metrics and logs 
- 11 Restore the service 
- 12 Calculate the SLO impact 
- 13 Write a short postmortem 



FOUR GOLDEN SIGNALS



LATENCY

How long requests take.

Goal: Keep it low and consistent.



TRAFFIC

How many requests are coming in.

Goal: Understand demand.



ERRORS

Rate of failed requests.

Goal: Keep errors as close to 0 as possible.



SATURATION

How full your resources are.

Goal: Leave headroom before reaching capacity.



SIMULATE FAILURE EXAMPLES

- High CPU / Memory usage
- Application crashes or restarts
- Slow database
- Dependency/API timeout
- Network issues



USEFUL TOOLS



Docker



Linux



Prometheus



Grafana



Alertmanager



Logs



SLO EXAMPLE

SLO: 99% Availability (30-day rolling window)

Total time in 30 days = 43,200 minutes

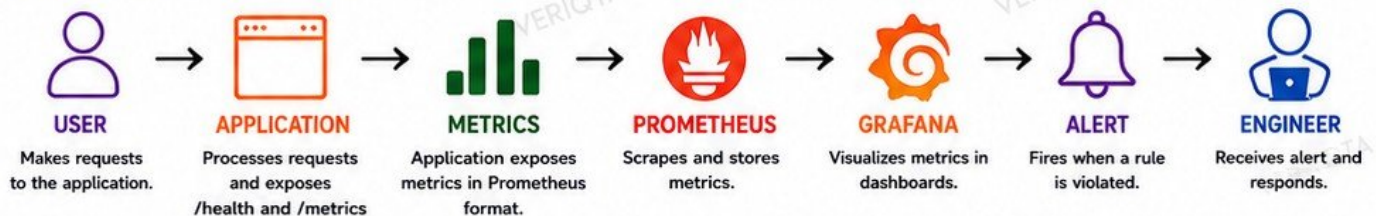
Allowed downtime (1%) = 432 minutes

If downtime = 86 minutes

SLO Achieved ☒ (86 < 432)



FINAL ARCHITECTURE



WHAT YOU WILL LEARN

- ☒ How to build and deploy a service
- ☒ How to make a service observable
- ☒ How to monitor real production signals
- ☒ How to detect and respond to incidents
- ☒ How to troubleshoot using data
- ☒ How to improve reliability continuously



SRE MINDSET

- ☒ Observe everything
- ☒ Focus on reliability
- ☒ Learn from every incident
- ☒ Improve a little every day



VERIQTA YouTube



VERIQTA GitHub



VERIQTA LinkedIn



VERIQTA X



VERIQTA Instagram



VERIQTA Telegram



veriqta.com

